International Collegiate Programming Contest

2026

Gran Premio de México 2026 - Primera Fecha

April 18th, 2026

Contest Session

This problem set contains 12 problems; pages are numbered from 1 to 16.

General information

Unless otherwise stated, the following conditions hold for all problems.

Program name

1. Your solution must be called *codename*`.c`, *codename*`.cpp`, *codename*`.java`, *codename*`.kt`, *codename*`.py3`, where *codename* is the capital letter which identifies the problem.

Input

1. The input must be read from standard input.
2. The input consists of a single test case, which is described using a number of lines that depends on the problem. No extra data appear in the input.
3. When a line of data contains several values, they are separated by *single* spaces. No other spaces appear in the input. There are no empty lines.
4. The English alphabet is used. There are no letters with tildes, accents, diaereses or other diacritical marks (ñ, Ã, é, Ì, ô, Ü, ç, etcetera).
5. Every line, including the last one, has the usual end-of-line mark.

Output

1. The output must be written to standard output.
2. The result of the test case must appear in the output using a number of lines that depends on the problem. No extra data should appear in the output.
3. When a line of results contains several values, they must be separated by *single* spaces. No other spaces should appear in the output. There should be no empty lines.
4. The English alphabet must be used. There should be no letters with tildes, accents, diaereses or other diacritical marks (ñ, Ã, é, Ì, ô, Ü, ç, etcetera).
5. Every line, including the last one, must have the usual end-of-line mark.
6. To output real numbers, round them to the closest rational with the required number of digits after the decimal point. Test case is such that there are no ties when rounding as specified.

Problem A – Anxiety at the restaurant

Margot Finch goes out to eat. A lot. Like, way too much. Birthdays, Fridays, random Wednesdays — she always finds a reason. And every time, the same thing happens: everyone at the table does their own math, puts their money in the middle, and walks out like everything is fine.

It is never fine.

The waiter always catches her at the door. Every. Single. Time. "Hey, we're still a little short." And because Margot cannot handle the embarrassment, she pays the difference. But here's the thing — paying the difference also makes her anxious. Now she's the one who paid more than everyone else, and that's awkward too. She replays the moment in her head the whole ride home.

And then one night she got home and realized the group had left the waiter zero tip. Zero pesos. The guy had been running back and forth all night. She thought about it for four days straight.

Look — Margot knows tips are not mandatory. She knows waiters should just earn a decent salary. She agrees! But her anxiety does not care about labor policy. If she doesn't leave at least 10

So now she wants a program. Something she can check before anyone stands up from the table. Something that tells her: are we good, or is this about to be a whole thing?

Input

The first line contains two integers N and M ($1 \leq N \leq 10^4,\ 1 \leq M \leq 10^3$): the number of items on the bill and the number of friends at the table (counting Margot).

The second line contains N integers $p_1, p_2, \ldots, p_N (1 \leq p_i \leq 10^4)$, the price of each item, rounded up to the nearest whole number.

The third line contains M integers $a_1, a_2, \ldots, a_M (1 \leq a_j \leq 10^4)$: the amount paid by Margot and her friends, also rounded up.

Output

If the total collected is greater than or equal to the bill plus 10% tip (rounded up), print "YES" meaning that all is good and Margot can relax

Otherwise, print: "NO" meaning that all is bad and Margot can't relax.

Sample input 1	**Sample output 1**
5 4 180 250 120 300 150 200 180 150 220	NO

Sample input 2	**Sample output 2**
4 5 200 350 250 200 220 210 200 190 230	NO

Sample input 3	**Sample output 3**
3 3 300 350 250 350 400 260	YES

Problem B – Bad LaTeX

Tomorrow is the Gran Premio de Mexico 2026 Primera Fecha, and you are in a hurry to finish setting all the problems for tomorrow's contest. Since two heads are better than one, you asked your friend Jimbo to help you out with the statements of the problems. However, this quote didn't end up being true — Jimbo wrote the statements but without respecting some writing style rules!

For an easier read, you want to ensure that the following rules are held:

- If an integer value is a power of 10 with $k \geq 4$ zeros, you write them as a power of 10 instead of its whole representation.

  For example, if Jimbo wrote 1000000000, then you must transform it to 10^9 **in LaTeX**, so the final text would be `10^{9}`.

- If an integer value ends with $k \geq 4$ zeros, you write them in an special scientific notation: You write a truncated real equivalent of the value with only 1 digit in the integral part and then multiply this real by the corresponding power of ten.

  For example, if Jimbo wrote 123400000, then you must transform it to $1.234 \cdot 10^8$ **in LaTeX**, so the final text would be `1.234\cdot10^{8}`

- These rules only apply to **integers that are not subscripts or superscripts or affected by any of them**. Any other value is kept as it is.

Since you are very tired from setting up the solutions, validators and testcases, you will automatize this process by writing a program that applies these rules on the statements.

Input

The first line of input contains an integer n ($1 \leq n \leq 100$) — The number of lines of a given statement.

The following n lines of input contain a string s_i ($1 \leq |s_i| \leq 1000$) – The i-th line contains the i-th line of the given statement.

It is guaranteed that the statement will only consist of letters (could be in lowercase or uppercase), digits, spaces and special symbols (`!?.,;$#{}_=+*`) and there won't be any leading or trailing space. Consider that if a line ends with an integer and the next one starts with an integer, these two are not part of the same value.

Output

Print n lines — The i-th line must contain the i-th line of the statement after applying the rules.

Sample input 1

```
4
This example shows a value
Of 1000000000 without being compressed to 10^{9}
Which is annoying when read. $ S_{10} = 2^{100000} + 780000 $
My ID is RA180000 but that was back in the year 20000
```

Sample output 1

```
This example shows a value
Of 10^{9} without being compressed to 10^{9}
Which is annoying when read. $ S_{10} = 2^{100000} + 7.8\cdot10^{5} $
My ID is RA180000 but that was back in the year 2\cdot10^{4}
```

Problem C – Cactus Simple Path Queries

You are given a connected, undirected, simple cactus graph. A cactus graph is a connected graph in which every edge belongs to at most one simple cycle. Equivalently, any two simple cycles have at most one common vertex.

Each edge has an integer weight, and weights may be negative.

Then you have to answer q queries. In each query, two vertices u and v are given, and you must find the minimum possible total weight of a **simple path** from u to v.

Input

The first line contains three integers n, m, and q ($2 \le n \le 200'000$, $n - 1 \le m \le 300'000$, $1 \le q \le 200'000$).

Each of the next m lines contains three integers u_i, v_i, and w_i ($1 \le u_i, v_i \le n$, $u_i \ne v_i$, $-10^9 \le w_i \le 10^9$), meaning that there is an undirected edge between u_i and v_i with weight w_i.

It is guaranteed that the graph is simple, connected, and is a cactus.

Each of the next q lines contains two integers u and v ($1 \le u, v \le n$), describing one query.

Output

For each query, print one integer: the minimum possible total weight of a simple path from u to v.

Sample input 1	**Sample output 1**
5 5 3 1 2 5 2 3 -10 3 4 5 4 1 1 3 5 2 1 3 5 4 2 4	-5 -2 -5

Notes

There are two simple paths from 1 to 3:

- $1 \to 2 \to 3$, with total weight $5 + (-10) = -5$;
- $1 \to 4 \to 3$, with total weight $1 + 5 = 6$.

So the answer to the first query is -5.

Problem D – Door 1

Pick a door.

The door you choose will determine your fate for the next N hours.

4, 3, 2, 1...

Door 1, "the ferret burrow".

You find yourself inside a vast underground labyrinth: tunnels branching in every direction, narrow passages, hidden chambers, and faint scratching sounds echoing through the walls. The air is warm, dense, and carries a faint earthy smell. In your hand, you carry a lamp.

You are not alone. This is a burrow inhabited by ferrets.

Some ferrets are harmless, timid creatures that scatter at the slightest noise. Others are not. There are also *light-sensitive ferrets*: if they spot you wandering through the dark tunnels, they will rush toward you unless you scare them away with your lamp. But the most dangerous of all is the *giant ferret*: silent, fast, and impossible to escape if you encounter it outside your hiding place.

Your lamp requires **exactly two** batteries to work, and both are consumed when it is used.

You carry a small bag that can hold **at most** K batteries. Initially, you have S batteries.

Every hour, you must choose **exactly one** action:

- **Hide.** You stay still in a safe chamber. No ferret can attack you while you are hiding.
- **Search for a battery.** You explore the tunnels, hoping to find a usable battery. During the i-th hour, you find **exactly one** battery with probability b_i.
- **Search for food.** You look for something edible. During the i-th hour, you find food with probability c_i. However, not all food is safe: if you find food, you die from poisoning with probability v_i. If you eat and survive, your hunger is reset.

Being outside your hiding place exposes you to the ferrets.

During the i-th hour, a *light-sensitive ferret* appears with probability q_i. If that happens while you are searching for a battery or food, you must use your lamp to survive. If you have fewer than two batteries before that hour, the ferret captures you, and you die. If you use your lamp, exactly two batteries are consumed.

The *giant ferret* appears during the i-th hour with probability p. Unfortunately, you do not know the exact value of p, but you know that it was chosen uniformly at random from $[0, 1]$ before you entered the ferret burrow, and that it remains fixed for all N hours. If it appears while you are searching for a battery or food, you are captured with certainty.

At the beginning of each hour, you must choose your action **before** knowing whether a *light-sensitive ferret* or the *giant ferret* will appear during that hour. If you choose to hide, you will still know whether the *giant ferret* appeared during that hour, although it cannot attack you.

Staying in these tunnels is exhausting. You must successfully eat at least once every H hours. Otherwise, you die from exhaustion.

If you manage to survive for N hours, you can escape.

You chose door 1. What is the maximum probability of survival if you act optimally?

Input

The first line contains four integers N, K, S and H ($1 \le N \le 500$, $1 \le K, H \le 12$ and $0 \le S \le K$).

The i-th of the next N lines contains four real numbers b_i, c_i, v_i and q_i ($0 \le b_i, c_i, v_i, q_i \le 1$). These numbers are given with **exactly** four digits after the decimal point — the probabilities for the i-th day.

Output

In a single line, print a real number — the maximum probability of survival if you act optimally.

The answer will be considered correct if its relative or absolute error doesn't exceed 10^{-9}.

Sample input 1	**Sample output 1**

`2 12 12 1`
`0.0000 0.0000 0.0000 0.0000`
`0.0000 0.0000 0.0000 0.0000`
	`0.0000000000`

Sample input 2	**Sample output 2**

`2 12 12 2`
`0.0000 0.0000 0.0000 0.0000`
`0.0000 0.0000 0.0000 0.0000`
	`0.0000000000`

Sample input 3	**Sample output 3**

`2 12 12 3`
`0.0000 0.0000 0.0000 0.0000`
`0.0000 0.0000 0.0000 0.0000`
	`1.0000000000`

Sample input 4	**Sample output 4**
`1 12 1 1` `1.0000 1.0000 0.5000 0.5000`	`0.1250000000`

Sample input 5	**Sample output 5**

`5 12 3 3`
`0.0123 0.4567 0.8901 0.2345`
`0.4567 0.8901 0.2345 0.0123`
`0.8901 0.2345 0.0123 0.4567`
`0.2345 0.0123 0.4567 0.8901`
`0.0123 0.4567 0.8901 0.2345`
	`0.1158078250`

Sample input 6	**Sample output 6**

`4 12 12 2`
`0.1234 0.0000 0.0000 0.0000`
`0.4321 1.0000 0.5000 0.0000`
`0.5678 1.0000 0.0000 1.0000`
`0.8764 0.0000 0.0000 0.0000`
	`0.1666666667`

Problem E – Erasmus Valthron

Erasmus Valthorn is the Chief Archivist of the Celestial Library of Numeris — a vast hall where every integer from 1 to N exists as a physical tome, its spine engraved with the number it represents.

After decades of cataloguing by mere numeric order, Erasmus has grown restless. He believes that numbers have a true identity — not their face value, but their prime essence: the sorted sequence of prime factors that compose them.

He calls this the Canonical Form of a number. For example:

$48 = 2 \times 2 \times 2 \times 2 \times 3 \quad \Rightarrow \quad [2, 2, 2, 3]$

$9 = 3 \times 3 \quad \Rightarrow \quad [3, 3]$

Erasmus now wishes to rearrange all tomes according to their Canonical Form, sorted lexicographically. Under this order, the tome of 48 precedes the tome of 9, since the first element of its sequence — 2 — is less than 3.

The number 1, having no prime factors, is represented by the empty sequence [ ], which is lexicographically smallest of all, and thus occupies the very first shelf.

However, Erasmus is old and forgetful. He keeps losing track of where specific tomes end up in the new arrangement. He has written down Q questions — each asking: "In this new order, which number sits at position K?"

Help Erasmus answer his queries before the next lunar eclipse!

Input

The first line contains two integers N and Q ($1 \leq N \leq 10^6,\ 1 \leq Q \leq 10^5$), the number of tomes in the library and the number of queries Erasmus has written down.

Each of the next Q lines contains a single integer K_i ($1 \leq K_i \leq N$), a position in the sorted arrangement.

Output

For each query, print a single integer — the number occupying position K_i in Erasmus's new ordering.

Sample input 1	**Sample output 1**
10 10 1 2 3 4 5 6 7 8 9 10	1 2 4 8 6 10 3 9 5 7

Sample input 2	**Sample output 2**
10 4 1 4 7 10	1 8 3 7

Sample input 3	**Sample output 3**
5 1 5	5

Problem F – F(x,l,r)

You are given an array $[a_1, a_2, \ldots, a_n]$ of length n and an integer k.

In one operation, you can choose any integer y and a range $[l, r]$, such that $1 \le l \le r \le n$ and $r - l + 1 \le k$, and assign $a_i := y$ for all $i \in [l, r]$.

Let's define $F(x, l, r)$ as the minimum number of operations to make $a_i \le x$ for all $i \in [l, r]$. Find the following sum:

$$\sum_{x=1}^{n} \sum_{l=1}^{n} \sum_{r=l}^{n} F(x, l, r)$$

As the answer can be very large, print it modulo 998244353.

Input

Each test contains multiple test cases. The first line contains the number of test cases t ($1 \le t \le 5 \cdot 10^4$). The description of the test cases follows.

The first line of each test case contains two integers n and k ($1 \le k \le n \le 3 \cdot 10^5$).

The second line of each test case contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le n$).

It is guaranteed that the sum of n over all test cases does not exceed $3 \cdot 10^5$.

Output

Print a single integer — the answer modulo 998244353.

Sample input 1	**Sample output 1**
4 3 2 1 1 1 3 1 1 2 3 3 3 3 3 3 12 4 3 1 4 1 5 9 2 6 5 3 5 9	0 10 12 648

Problem G – Gerald the mudcrab

Deep in the marshes of Vvardenfell, there is a mudcrab. Not just any mudcrab — the mudcrab. The one with 10,000 gold and nothing to sell. Travelers come from across Morrowind just to stare at him.

His name, as far as anyone can tell, is Gerald.

Gerald has recently decided to expand his business. He has acquired exactly N soul gems, each engraved with a number by some bored Telvanni wizard. A complete collection means having exactly one gem for each number from 1 to N no duplicates, no missing numbers. The order on the shelf doesn't matter. Gerald just wants one of each.

The problem is the wizard was drunk. Some numbers were engraved twice. Some were skipped entirely. Gerald now has N soul gems, but the numbers are a mess.

Gerald can trade with a merchant in Balmora giving one of his gems for any other gem. But every trip through the swamp is exhausting, and Gerald, despite being a successful business-crab, would rather minimize travel.

Help Gerald figure out the minimum number of trades needed so that his collection contains exactly one gem of each number from 1 to N.

Input

The first line contains a single integer N ($1 \leq N \leq 10^5$): the number of soul gems Gerald owns.

The second line contains N integers $a_1, a_2, \ldots, a_N (1 \leq a_i \leq N)$: the number engraved on each soul gem.

Output

Print a single integer: the minimum number of trades needed so that Gerald has exactly one gem for each number from 1 to N. The gems do not need to be in any particular order — Gerald just needs one of each.

Sample input 1	**Sample output 1**
5 1 2 3 4 5	0

Sample input 2	**Sample output 2**
5 1 1 2 4 4	2

Sample input 3	**Sample output 3**
3 3 2 2	1

Problem H – Hidden Symmetry of Valdris

Deep beneath the Citadel of Eternal Letters lies the Whispering Archives — a vault sealed for millennia, whose enchanted doors open only to those who can recite two simultaneous palindromic incantations.

The vault's guardian, Valdris the Scribe, inscribed its secrets across two sacred scrolls: A and B. To unseal the vault, a supplicant must pick a split point i and a meeting index j, then chant both of the following at once:

$P_1 = A[1 \dots i] + B[j \dots |B|]$

$P_2 = B[1 \dots i] + A[j \dots |A|]$

Both P_1 and P_2 must be palindromes simultaneously. Centuries of failed attempts have filled the antechamber with very flat scholars. Help the next adventurer survive by counting every valid pair (i, j) such that $1 \le i, j \le |A|$.

Input

The first line contains T ($1 \le T \le 10^5$).

Each test case consists of two lines containing strings A and B of lowercase English letters, with $|A| = |B|$ ($1 \le |A|, |B| \le 10^6$).

Sum of $|A| + |B|$ over all test cases is less than 2×10^6

Output

For each test case, a single integer — the number of valid pairs (i, j).

Sample input 1	**Sample output 1**
4 abacaba abacaba aaa aaa abc cba abyyycd dczzzba	25 9 7 10

Notes

A string S of length ℓ is a palindrome if $S[k] = S[\ell - k + 1]$ for all $1 \le k \le \ell$. The empty string is considered a palindrome.

Problem I – Inner Product

You are given a sequence of positive integers $a_1, a_2, \ldots, a_n$ and a string s of length $n-1$.

For each i from 1 to $n-1$:

- if s_i is equal to $<$, then $b_i < b_{i+1}$;
- if s_i is equal to $=$, then $b_i = b_{i+1}$;
- if s_i is equal to $>$, then $b_i > b_{i+1}$.

Your task is to choose a sequence of positive integers $b_1, b_2, \ldots, b_n$ satisfying all these relations.

Among all valid sequences, minimize

$$a_1 b_1 + a_2 b_2 + \cdots + a_n b_n.$$

For the given constraints, it can be proved that the optimal sequence is unique.

Input

The first line contains one integer n ($2 \le n \le 200'000$).

The second line contains n integers $a_1, a_2, \ldots, a_n$ ($1 \le a_i \le 1'000'000$).

The third line contains a string s of length $n-1$, consisting only of the characters `<`, `=`, and `>`.

Output

Print the minimum possible value of

$$a_1 b_1 + a_2 b_2 + \cdots + a_n b_n$$

in the first line.

In the second line, print the unique optimal sequence $b_1, b_2, \ldots, b_n$.

It can be proved that, for the given constraints, the minimum value always fits in a signed 64-bit integer.

Sample input 1	**Sample output 1**
6 3 1 4 1 5 9 <=>><	43 1 3 3 2 1 2

Sample input 2	**Sample output 2**
2 2 7 <	16 1 2

Sample input 3	**Sample output 3**
8 8 2 2 8 3 7 4 6 =<<=>>=	71 1 1 2 3 3 2 1 1

Notes

The first sample requires

$$b_1 < b_2 = b_3 > b_4 > b_5 < b_6.$$

The sequence 1 3 3 2 1 2 satisfies all relations, and its cost is

$$3 \cdot 1 + 1 \cdot 3 + 4 \cdot 3 + 1 \cdot 2 + 5 \cdot 1 + 9 \cdot 2 = 43.$$

Problem J – Just the right enchantment

Seraphina Coldwater is the most celebrated pastry chef of the Arcane Confectionery Guild. Her legendary cakes are not merely delicious — they are magical. Each cake requires exactly three ingredients, chosen from her enchanted pantry, where every ingredient is labeled with an enchantment level from 1 to N.

However, Seraphina is a perfectionist. She insists that a cake is only worth baking if its total enchantment — the sum of the three chosen ingredients' levels — is an even number. An odd total, she claims, makes the frosting curdle and the candles flicker in an unsettling fashion.

Being a meticulous record-keeper, Seraphina wants to know exactly how many distinct valid combinations she can bake. Since she respects each ingredient's individuality, she always picks three different ingredients, and she considers two selections identical if they contain the exact same trio regardless of order.

Given the size of her pantry N, help Seraphina count the number of valid triples (a, b, c) with $1 \leq a < b < c \leq N$ such that $a + b + c$ is even.

As pantries can grow astronomically large, the answer may be an enormous number. Seraphina's enchanted ledger can only record values modulo 10^9+7 — print the answer modulo $10^9 + 7$.

Input

The first line contains a single integer T ($1 \leq T \leq 10^5$), the number of pantry configurations Seraphina wishes to evaluate.

Each of the next T lines contains a single integer N ($1 \leq N \leq 10^6$), the number of ingredients available.

Output

For each test case, print a single integer — the number of valid magical cake combinations, modulo $10^9 + 7$.

Sample input 1	**Sample output 1**
3 3 4 7	1 2 19

Sample input 2	**Sample output 2**
2 100 1000	80850 83083500

Sample input 3	**Sample output 3**
1 3	1

Problem K – Kernel of the Disks

You are given a point P and n closed disks on the plane.

Disk i is centered at (x_i, y_i) and has radius r_i. It contains all points (x, y) such that

$$(x - x_i)^2 + (y - y_i)^2 \leq r_i^2.$$

It is guaranteed that the intersection of all n disks is non-empty.

Find the minimum possible Euclidean distance from P to a point that belongs to all disks.

Input

The first line contains one integer n ($1 \leq n \leq 500$).

The second line contains two integers p_x and p_y ($-1'000'000 \leq p_x, p_y \leq 1'000'000$), the coordinates of P.

Each of the next n lines contains three integers x_i, y_i, r_i ($-1'000'000 \leq x_i, y_i \leq 1'000'000$, $1 \leq r_i \leq 1'000'000$).

Output

Print one real number: the minimum distance from P to the intersection of all disks.

Your answer will be considered correct if its absolute or relative error does not exceed 10^{-6}.

Sample input 1	**Sample output 1**
3 -3 -4 1 0 1 0 1 1 0 0 10	5.000000000000000

Notes

The first two disks intersect in a lens whose boundary passes through $(0, 0)$ and $(1, 1)$. The third disk contains that whole lens.

The closest point in the common intersection to $P = (-3, -4)$ is $(0, 0)$, so the answer is 5.

Problem L – Legendary Sort

The Legendary Huron is learning about sorting algorithms. After studying Bubble Sort and Bogo Sort, he became so enthusiastic that he invented his own algorithm: the *Legendary Sort*. This algorithm sorts an array and consists of two phases:

1. First, rotate the array to the left by k positions ($0 \le k < n$). Formally, the array $[a_1, a_2, \ldots, a_n]$ becomes $[a_{k+1}, a_{k+2}, \ldots, a_n, a_1, a_2, \ldots, a_k]$. For example, the array $[1, 2, 3, 4, 5]$ becomes $[3, 4, 5, 1, 2]$ when $k = 2$.
2. Then, while the array is not sorted, repeatedly perform the following operation: find an index i ($1 \le i < n$) such that $a_i > a_{i+1}$ and swap these two elements. If there are multiple valid indices i, any of them can be chosen.

The Legendary Huron wants to test his new algorithm in different scenarios. For that, he considers a **permutation** $[p_1, p_2, \ldots, p_n]$ of length n, and processes q queries on this permutation. Each query is one of the following types:

1. Given an index i ($1 \le i < n$), swap the elements p_i and p_{i+1}.
2. Given an integer k ($0 \le k < n$), execute the Legendary Sort on the current permutation, rotating it by k positions to the left in the first phase. Compute the minimum number of swaps that can be performed in the second phase. Note that the rotation of the first phase **does not** persist for future queries.

Help the Legendary Huron to process these queries.

Input

The first line contains two integers n and q ($2 \le n \le 2 \cdot 10^5$ and $1 \le q \le 2 \cdot 10^5$) — the length of the permutation and the number of queries.

The second line contains n **distinct** integers $p_1, p_2, \ldots, p_n$ ($1 \le p_i \le n$) — the permutation.

Each of the next q lines describes a query. Each line begins with an integer t ($t \in \{1, 2\}$) — the type of the query. If $t = 1$, an integer i ($1 \le i < n$) follows. If $t = 2$, an integer k ($0 \le k < n$) follows.

Output

For each query of type $t = 2$, print an integer — the minimum number of times that the swap operation can be performed in the second phase.

Sample input 1	**Sample output 1**
5 5 1 2 3 4 5 2 0 2 2 1 2 2 0 2 2	0 6 1 5

Sample input 2	**Sample output 2**
5 2 3 4 5 1 2 2 3 2 0	0 6